# Less context, more accuracy

A voice scheduler that asks small tools instead of reading the whole catalog

82–96%

tests passed
(full catalog: 24–67%)

22–33k → 27–31k

prompt tokens per conversation,
50 → 300 providers
(full catalog: 80–91k → 379–515k)

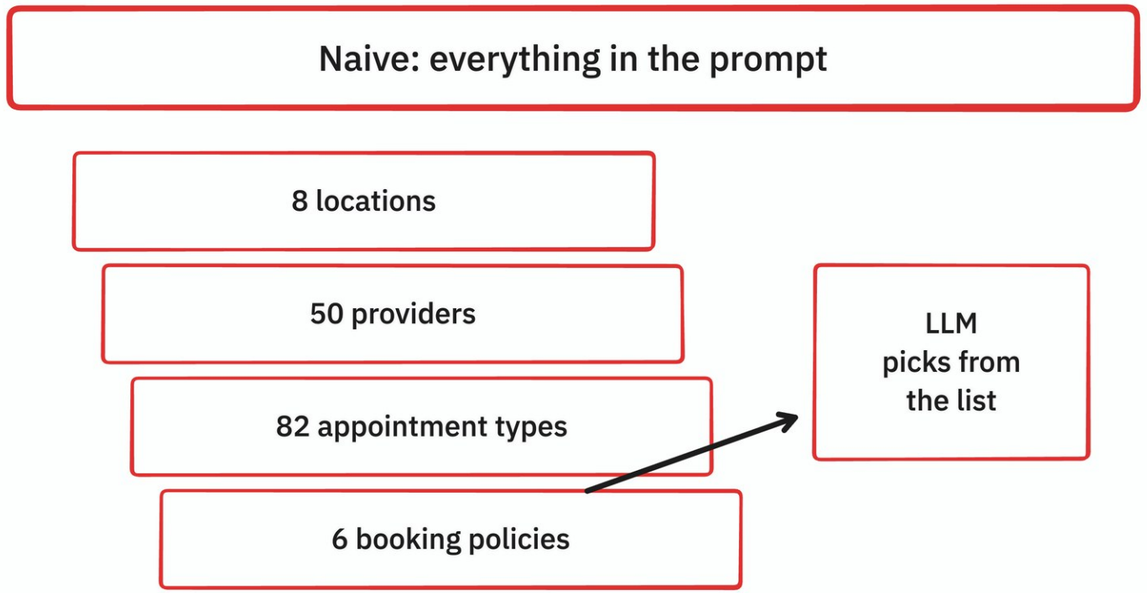
$0.051

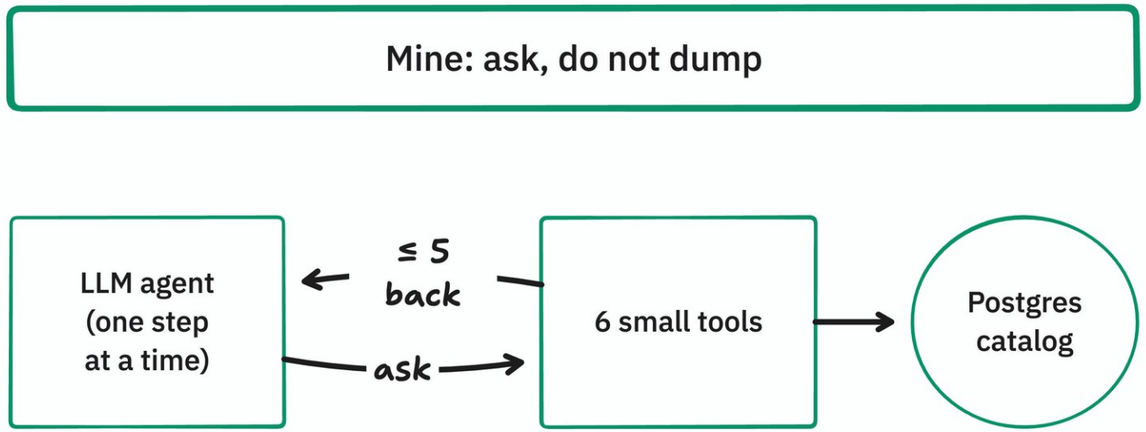
per conversation with gpt-4o
(full catalog: $0.111)

The agent that sees less is right more often. Cost and accuracy move together.

# The problem: one long list

Dumping the catalog into the prompt fails twice, and the second failure is worse




- Cost: the prompt grows with the catalog.
- Accuracy: the model picks the wrong one of 82 similar names.
- At 300 providers, gpt-4o passes 10% of the tests.

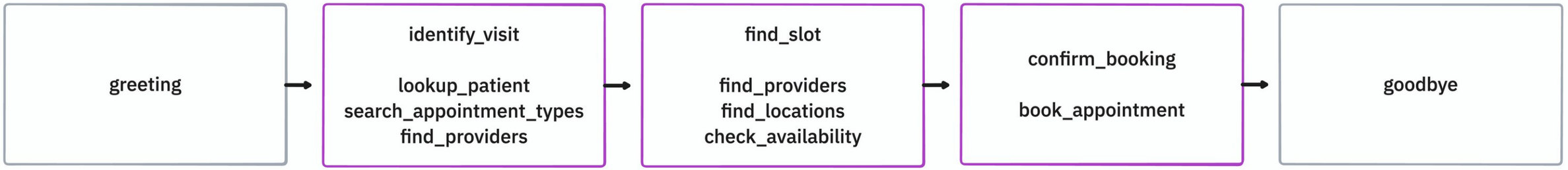
- The prompt stays the same size for any catalog.
- The tools do the hard, exact work.
- At 300 providers, the same agent passes 75%.

# Code for facts, models for meaning

| Code (deterministic, 100% correct) | Models (fuzzy, natural language) |
| --- | --- |
| Who works where, who offers what | Caller words → visit type<br>(embeddings + Decisions API) |
| Booking policies (referral, new patient, capability) | The conversation itself (LLM agent) |
| Free slots: hours × bookings × duration | Asking the right question when unclear |
| Distances between places (PostGIS) | |
| Names that sound alike (trigram, phonetic) | |

# Agent graph and tool contract

Each step sees only its own tools. Each tool answers with a status and an instruction.




+ answer_questions (find_providers, find_locations, search_appointment_types) from greeting

```
{
  "status": "match",
  "total": 10, "shown": 5, "truncated": true,
  "options": [ { "name": "Full Body Skin Exam", ... }, ... ],
  "instruction": "The first option is the match.
                  Confirm it in plain words."
}
```

- status: each tool has its own (the type search: match · ask · unsure · not_offered · no_match). The agent knows at once what happened.

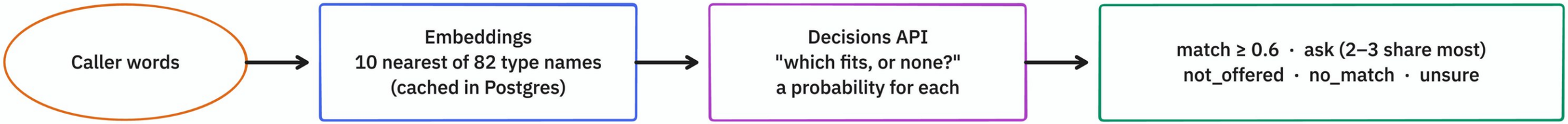
- instruction: one sentence with the next step. The agent does not have to work it out.

- max 5 options + total: the context stays small.

- Bad arguments ("Dr. Nguyen" for an id) are resolved or explained, never an empty list.

# Find the visit type: embeddings filter, Decisions API decides

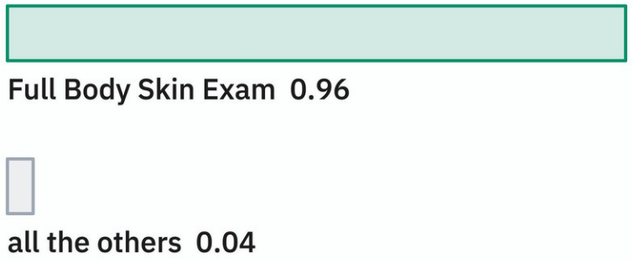
Callers say "my skin", the catalog says "Full Body Skin Exam"




## Example (measured): "I want my skin checked"

Shortlist (embeddings):
1. Full Body Skin Exam
2. Skin Cancer Screening
3. Dermatology Consultation
4. Allergy Skin Testing
5. Mole Removal
... 5 more

Decisions API:




status: match
Agent: "A full body skin exam, is that right?"

Measured on lay phrases:

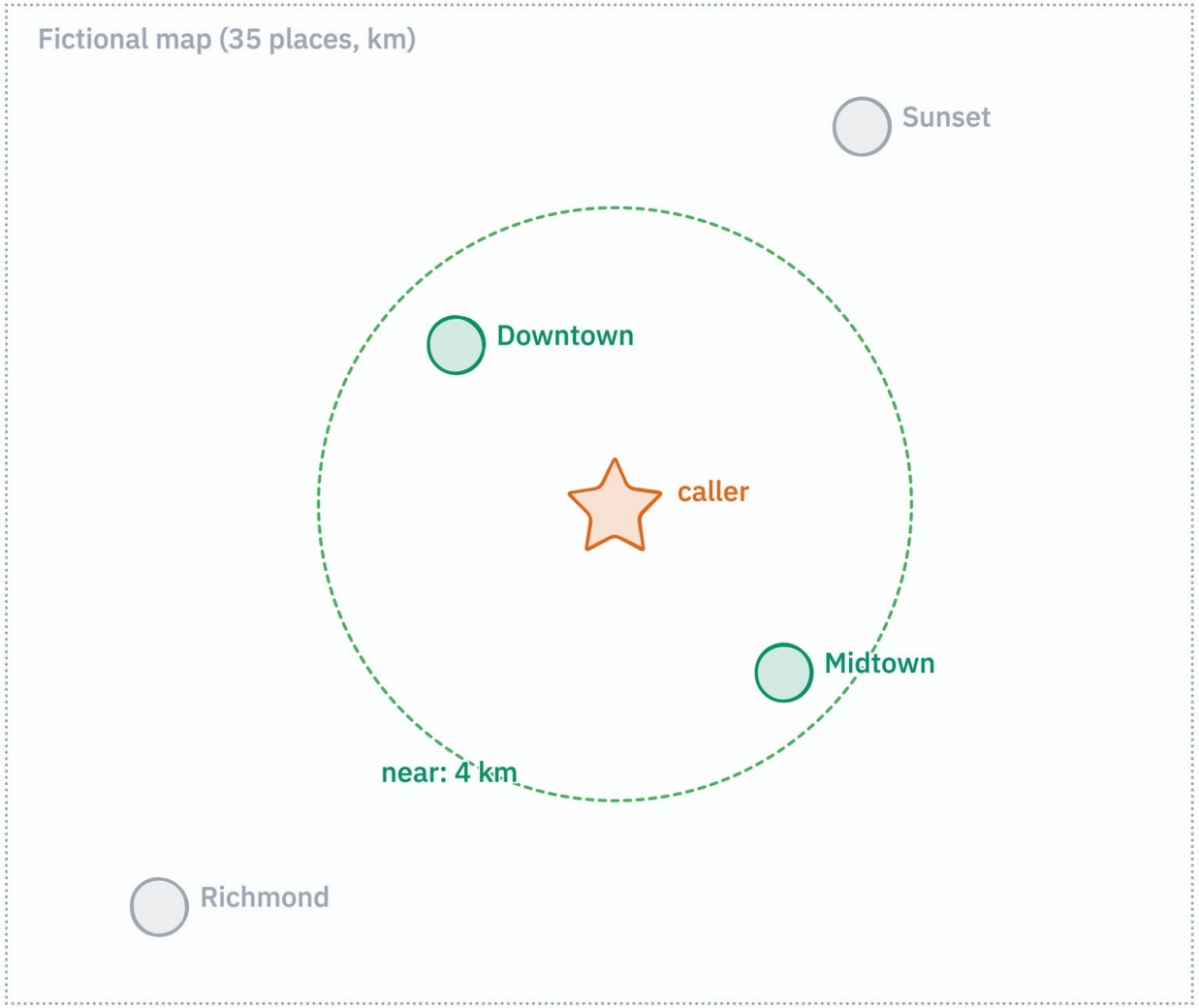
- new: 39/40 right, 32/32 on held-out phrases
- old (aliases + full text + vectors): 29/40 and 19/32

No hand-written lexicon per catalog. A new catalog only needs its type names embedded.

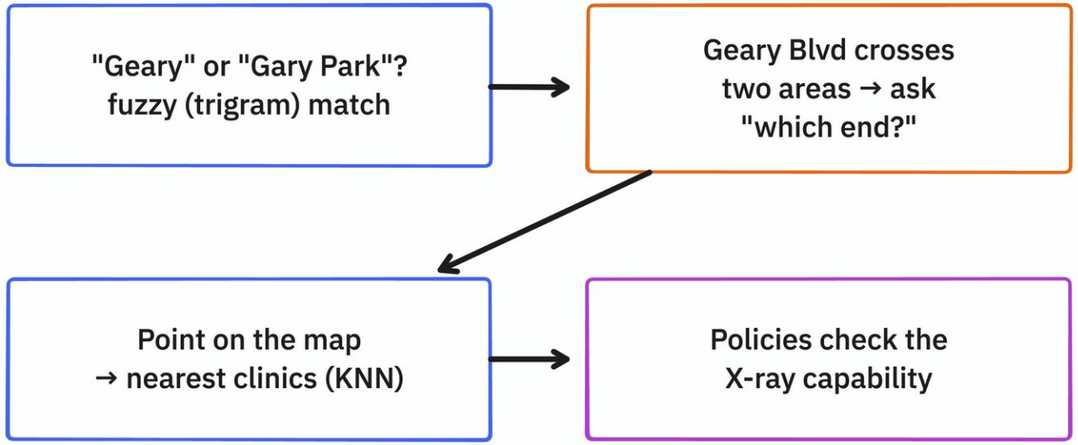
If embeddings fail: Decisions sees all types.
If Decisions fails: the agent asks, never guesses.

# Places: from spoken words to the nearest clinic

Callers say streets and areas, and speech-to-text gets them wrong



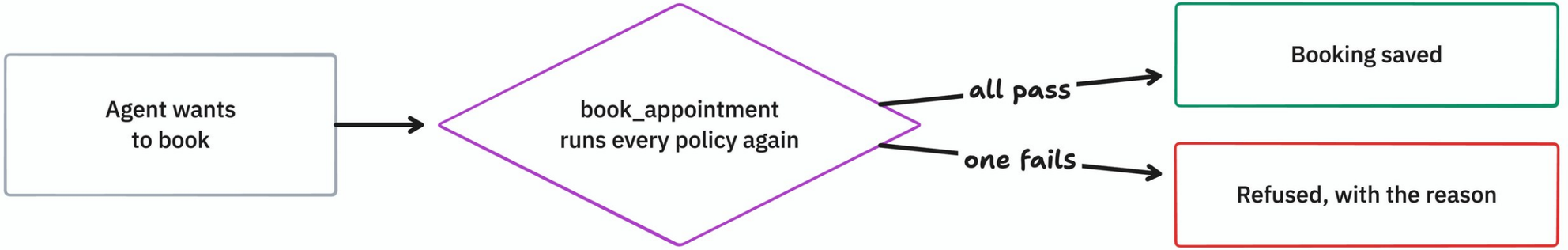

Example: "Hi, I need an X-ray, near Geary if possible."




- Distance ranks clinics. It never decides if a booking is allowed.
- If a name is unclear, the agent asks. It does not guess.

# Policies are code, with a last gate

## Booking rules must be right every time, so they are functions, not prompts



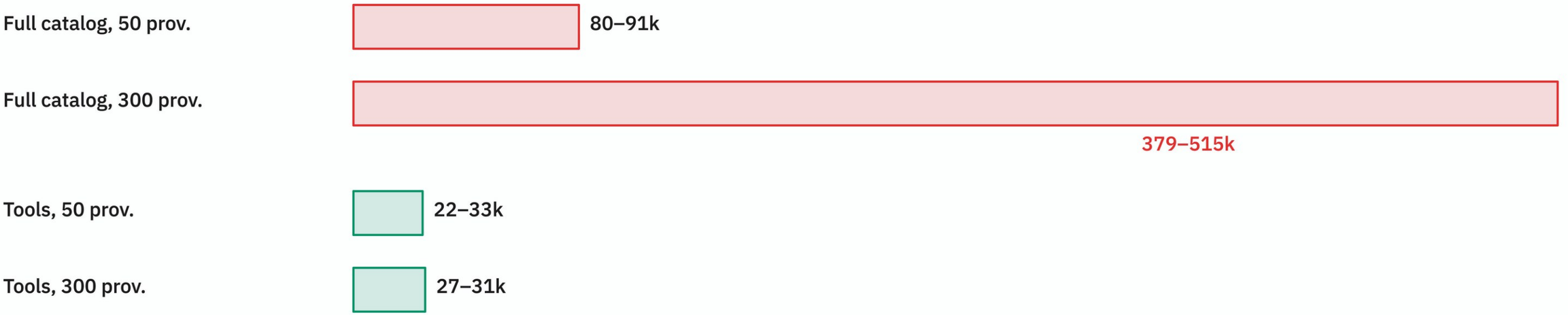

One function per rule (backend/store/policies.py), 6 policies, 7 checks:
- provider works at location
- provider offers type
- location has the capability
- referral on file
- type open to new patients
- provider accepting new patients
- provider at several locations: location settled with the caller

Why code, not an LLM judge:
- A judge also needs the true answer. Then use the answer directly.
- The same functions filter search results, gate bookings and give the reason the caller hears.
- New policy: hours of work with AI tools, and a test.

Evidence: when the model wrote SQL, it tried bad bookings. The gate saved 0 of them.

# It scales: the prompt does not grow

Prompt tokens per conversation, 50-provider catalog → 300-provider catalog




Tests passed at 300 providers

Full catalog: 10–25%
Tools: 75%

Why it holds:

- Fixed shortlist (10) and max 5 options per answer.
- The database does the search, with indexes (pgvector, PostGIS KNN, trigram).
- Embeddings are made once per catalog and cached.
- A new clinic = ingest its catalog. No prompt changes.

# Alternatives I tried and why I chose this

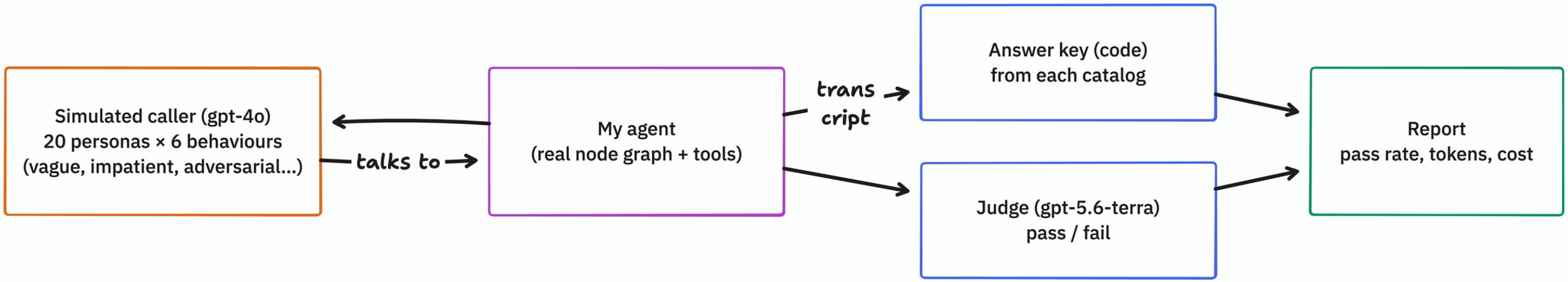
Every option was measured, not guessed

| Option | Result | Decision |
| --- | --- | --- |
| Full catalog in the prompt | 24–67% passed; 80k–515k tokens | Baseline only |
| Model writes SQL over the schema | 36% vs 100% (gpt-4o); 2–3× tool calls | Removed |
| Aliases + full text + vectors | 29/40 right; a lexicon per catalog | Replaced |
| Embeddings + Decisions API | 39/40 right; one API call; fallbacks | Kept |
| LLM judges the policies | Needs the true answer anyway | Code instead |
| Equipment as a model argument | Used 5 of 468 times, always in place of the visit | Removed (a rule in code) |

Latency is first on a phone call: one tool call does many checks, so the agent makes fewer round trips.

# How I know: an eval over the real agent

Text-mode calls through the same node graph and tools as the voice agent




- 10 catalogs: the given one, 6 generated (small, 300 providers, sparse, …), 3 with traps (twin doctors, near-duplicate names, referral maze).
- 4 models, and a full-catalog baseline in the same run.
- Every design change came from a failing conversation, and moved a number. Example: a swallowed error turned off vector search for repeated phrases. The eval found it.

# Scope: built, mocked, left out

Focus on the catalog, as the task asks. Mock what is not the problem.

## Built

- Builder UI: edit the graph, test call, live graph + tool calls
- 6 tools + agent graph
- Visit type search
- Places and fuzzy names
- Policies + booking gate
- Eval harness + baseline
- Monitoring: every call traced to Langfuse

## Mocked

- Calendar: seeded hours and bookings
- Patients and referrals
- Map: 35 fictional places
- 9 extra test catalogs

## Left out (on purpose)

- Voice quality: not the challenge
- ORM: plain SQL, every query visible
- Multiple clients: one clinic group at a time (tables keyed by catalog: a first step)
- Real calendar / EHR
- Policies for many clients (rules per clinic group)
- Audio in the evals